

Table of Contents

1. Java Ecosystem & Spring	2
Java SE	2
Java EE → Jakarta EE	2
Spring	3
IoC + DI + modular infrastructure	3
2. Spring Core: IoC & DI	3
The Problem: Tight Coupling	3
IoC — Inversion of Control	3
DI — Dependency Injection	4
IoC vs DI	4
3. Spring IoC Container & Beans	4
IoC Container	4
Bean	4
4. Bean Scopes	5
Singleton — Default	5
Prototype	5
Web Scopes	5
5. Important Spring Annotations	5
Stereotype Annotations	5
Dependency Injection	6
Configuration	6
Property Injection	7
6. Spring Boot	7
@SpringBootApplication	7
Starter Dependencies	7
Embedded Server	8
7. Spring MVC	8
8. DispatcherServlet	8
What is DispatcherServlet?	8
Important Components	9
Simplified Request Flow	9
9. Spring Data JPA & Hibernate	9
ORM	9
Spring Data JPA	10
JPA vs Hibernate vs Spring Data JPA	10
10. Important JPA Annotations	11
@Entity	11
@Table	11

@Id	11
@GeneratedValue	11
@Column	12
@Transient	12
11. Transactions	12
12. Spring Security	12
13. Lombok	13
14. Most Common Interview Questions	14
Q1. What is IoC?	14
Q2. What is DI?	14
Q3. IoC vs DI?	14
Q4. What is a Bean?	14
Q5. What is ApplicationContext?	14
Q6. Why is constructor injection preferred?	14
Q7. What is the default Bean scope?	14
Q8. Does Singleton mean one object for the entire JVM?	14
Q9. What is DispatcherServlet?	15
Q10. JPA vs Hibernate?	15
Q11. JPA vs Spring Data JPA?	15
Q12. What is ORM?	15
Q13. Why does an Entity need a no-arg constructor?	15
Q14. @Controller vs @RestController?	15
Q15. @Component vs @Service vs @Repository?	15
Q16. @Bean vs @Component?	15
Q17. @Value vs @ConfigurationProperties?	15
Q18. What does @Transactional do?	16
Q19. What problem does Spring Boot solve?	16
Q20. Does Spring Boot replace Spring?	16
15. One-Minute Mental Model	16

1. Java Ecosystem & Spring

Java SE

Core Java platform: OOP, Collections, Exceptions, Threads, I/O, etc.

Used for general-purpose Java applications.

Java EE → Jakarta EE

- **Java EE:** Enterprise Java specifications such as Servlets, JPA, JMS, etc.

- **Jakarta EE:** Successor to Java EE after Oracle transferred Java EE to the Eclipse Foundation.
- Package namespace changed from `javax.` to `jakarta.`

Spring

Spring is a framework for building Java applications.

Its core idea is:

IoC + DI + modular infrastructure

Spring can work with Jakarta EE technologies such as Servlet and JPA.

2. Spring Core: IoC & DI

The Problem: Tight Coupling

```
public class UserController {  
    private UserService userService = new UserService();  
}
```

`UserController` directly creates `UserService`.

Problems:

- Tight coupling
- Difficult testing
- Difficult replacement of implementations
- Object lifecycle is controlled by application code

IoC — Inversion of Control

IoC means **control of object creation and lifecycle is transferred from application code to the framework/container.**

Without Spring:

```
UserService service = new UserService();
```

With Spring:

```
@Service
```

```
public class UserService { }
```

Spring creates and manages the object as a **Bean**.

DI — Dependency Injection

DI is the mechanism through which dependencies are supplied to an object instead of the object creating them itself.

```
@RestController
public class UserController {
    private final UserService userService;
    public UserController(UserService userService) {
        this.userService = userService;
    }
}
```

Spring creates `UserService` and injects it into `UserController`.

IoC vs DI

- **IoC**: Principle — control is inverted.
- **DI**: Technique used to implement IoC by supplying dependencies.

3. Spring IoC Container & Beans

IoC Container

The Spring IoC container creates, configures, stores and manages Beans.

Main container interfaces:

- `BeanFactory`
- `ApplicationContext`

In Spring Boot, `ApplicationContext` is normally used.

Bean

A Bean is simply an object whose lifecycle is managed by the Spring IoC container.

```
@Component
public class EmailService { }
```

Spring creates and manages `EmailService` as a Bean.

4. Bean Scopes

Scope defines **how many instances of a Bean exist and how long they are associated with the container/context**.

Singleton — Default

One instance **per Spring container**.

```
@Service
public class PaymentService { }
```

Multiple injections normally reference the same instance.

Prototype

A new instance is created each time the Bean is requested from the container.

```
@Scope("prototype")
@Component
public class ReportGenerator { }
```

Important: injecting a prototype Bean into a singleton does **not** automatically create a new instance on every method call.

Web Scopes

- `request` → one Bean instance per HTTP request
- `session` → one Bean instance per HTTP session
- `application` → one instance per `ServletContext`

5. Important Spring Annotations

Stereotype Annotations

`@Component` Generic Spring-managed Bean.

`@Service` Specialized `@Component` for business/service logic.

`@Repository` Specialized `@Component` for persistence/DAO logic. Also enables Spring's exception translation for supported persistence technologies.

@Controller Used for MVC controllers that usually return views.

@RestController Equivalent to:

```
@Controller
@ResponseBody
```

Used for REST APIs where return values are written to the HTTP response body.

Dependency Injection

@Autowired Tells Spring to inject a matching Bean.

Can be used on:

- Constructor
- Setter
- Field

Best practice: prefer constructor injection.

```
@Service
public class OrderService {
    private final PaymentService paymentService;
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

If the class has a single constructor, **@Autowired** is usually unnecessary.

Configuration

@Configuration Marks a class as a source of Bean definitions.

@Bean Registers the object returned by the method as a Spring Bean.

```
@Configuration
public class AppConfig {
    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}
```

Useful especially for third-party classes that you cannot annotate with **@Component**.

Property Injection

`@Value` Reads a value from configuration.

```
@Value("${server.port}")  
private int port;
```

For larger configuration groups, `@ConfigurationProperties` is generally preferred.

6. Spring Boot

Spring Boot simplifies building Spring applications through:

- Auto-configuration
- Starter dependencies
- Embedded servers
- Convention over configuration
- Production-ready features

It is **built on top of Spring Framework**; it does not replace Spring.

`@SpringBootApplication`

Combines:

```
@Configuration  
@EnableAutoConfiguration  
@ComponentScan
```

Starter Dependencies

Example:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

This provides the common dependencies required for Spring web applications, including Spring MVC and an embedded servlet container.

Embedded Server

Spring Boot applications can run with an embedded server such as Tomcat.

Therefore, for a typical Boot application:

```
java -jar application.jar
```

is enough to start the server; Tomcat does not need to be installed separately.

7. Spring MVC

Spring MVC is Spring's web framework for handling HTTP requests.

Basic flow:

```
Client  
[]  
DispatcherServlet  
[]  
HandlerMapping  
[]  
Controller Method  
[]  
Response
```

For REST APIs:

```
@RestController  
@RequestMapping("/users")  
public class UserController {  
    @GetMapping  
    public List<User> getUsers() {  
        return userService.findAll();  
    }  
}
```

GET /users is mapped to `getUsers()`.

8. DispatcherServlet

What is DispatcherServlet?

`DispatcherServlet` is Spring MVC's **Front Controller**.

It receives HTTP requests and coordinates request processing.

Important Components

HandlerMapping

- Finds which controller/handler should handle the request.

HandlerAdapter

- Invokes the selected controller method.

HttpMessageConverter

- Converts request/response bodies between Java objects and formats such as JSON.

ViewResolver

- Resolves logical view names to actual views.
- Mainly relevant when returning server-side views, not typical REST APIs.

Simplified Request Flow

```
HTTP Request
[]
DispatcherServlet
[]
HandlerMapping
[]
HandlerAdapter
[]
Controller
[]
HttpMessageConverter
[]
HTTP Response
```

9. Spring Data JPA & Hibernate

ORM

ORM maps Java objects to relational database tables.

Example:

```
@Entity
public class User {
    @Id
```

```
private Long id;  
private String name;  
}
```

Conceptually:

User object ↔ users table

Spring Data JPA

Spring Data JPA provides repository abstractions so you do not have to write common CRUD code manually.

```
public interface UserRepository  
extends JpaRepository<User, Long> {  
}
```

You automatically get methods such as:

```
save()  
findById()  
findAll()  
deleteById()  
existsById()
```

These methods come from **Spring Data**, not from the JPA specification itself.

JPA vs Hibernate vs Spring Data JPA

- **JPA**: Specification/API for persistence.
- **Hibernate**: Popular implementation of JPA.
- **Spring Data JPA**: Spring abstraction that provides repository APIs and integrates JPA with Spring.

Relationship:

```
Spring Data JPA  
└─  
  JPA  
  └─  
    Hibernate  
    └─  
      Database
```

10. Important JPA Annotations

@Entity

Marks a class as a JPA entity.

```
@Entity
public class User { }
```

JPA requires a no-argument constructor (normally **public** or **protected**) for entity instantiation.

@Table

Specifies the database table.

```
@Entity
@Table(name = "users")
public class User { }
```

@Id

Marks the primary key.

```
@Id
private Long id;
```

@GeneratedValue

Defines how the primary key value is generated.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Common strategies:

- **IDENTITY** → database identity/auto-increment mechanism
- **SEQUENCE** → database sequence
- **AUTO** → provider chooses an appropriate strategy

Do **not** assume **AUTO** always creates a separate sequence table; the actual strategy depends on the JPA provider and database.

@Column

Customizes column mapping and constraints.

```
@Column(  
    name = "email",  
    nullable = false,  
    unique = true  
)  
private String email;
```

@Transient

Field is ignored by JPA persistence.

```
@Transient  
private int calculatedAge;
```

Useful for values calculated at runtime that should not be stored in the database.

11. Transactions

@Transactional

Runs a method/class inside a transaction managed by Spring.

```
@Transactional  
public void transferMoney() {  
    debit();  
    credit();  
}
```

If the transaction fails and rollback rules apply, the transaction is rolled back.

Important interview point:

- **@Transactional** works through Spring's transaction infrastructure/proxies.
- Self-invocation can prevent proxy-based transactional behavior from being applied.

12. Spring Security

Spring Security provides:

- Authentication → **Who are you?**
- Authorization → **What are you allowed to do?**

Example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() {
}
-
```

Common authentication/security mechanisms:

- Session-based authentication
- Basic Authentication
- OAuth 2.0 / OpenID Connect
- JWT-based authentication

Important:

JWT is a token format/technology, not an authentication framework by itself.

13. Lombok

Lombok generates boilerplate code at compile time.

Common annotations:

`@Getter` Generates getters.

`@Setter` Generates setters.

`@NoArgsConstructor` Generates no-argument constructor.

`@AllArgsConstructor` Generates constructor with all fields.

`@RequiredArgsConstructor` Generates constructor for `final` and `@NonNull` fields.

`@Data` Combines commonly used getters, setters, `equals`, `hashCode`, and `toString`.

For Spring services, `@RequiredArgsConstructor` is commonly used with constructor injection.

```
@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;
}
```

14. Most Common Interview Questions

Q1. What is IoC?

IoC is the principle of transferring object creation and lifecycle management from application code to the Spring container.

Q2. What is DI?

DI is the process of supplying an object's dependencies from outside instead of the object creating them itself.

Q3. IoC vs DI?

IoC is the **principle**; DI is a **mechanism/pattern** used to achieve it.

Q4. What is a Bean?

An object whose lifecycle is managed by the Spring IoC container.

Q5. What is ApplicationContext?

A central Spring IoC container that creates and manages Beans and provides additional application features such as events, resource loading, and message resolution.

Q6. Why is constructor injection preferred?

- Dependencies are explicit.
- Supports immutability with **final**.
- Easier unit testing.
- Prevents partially initialized objects.

Q7. What is the default Bean scope?

singleton — one instance per Spring container.

Q8. Does Singleton mean one object for the entire JVM?

No.

It means one instance **per Spring IoC container**.

Q9. What is DispatcherServlet?

Spring MVC's Front Controller that receives requests and coordinates request handling.

Q10. JPA vs Hibernate?

JPA is a specification; Hibernate is an implementation of that specification.

Q11. JPA vs Spring Data JPA?

JPA defines persistence APIs/specification.

Spring Data JPA provides repository abstractions such as `JpaRepository`.

Q12. What is ORM?

A technique for mapping Java objects to relational database tables.

Q13. Why does an Entity need a no-arg constructor?

JPA requires a no-argument constructor so the persistence provider can instantiate entity objects.

Q14. `@Controller` vs `@RestController`?

`@Controller` is typically used for MVC views.

`@RestController` is used for REST APIs and includes `@ResponseBody`.

Q15. `@Component` vs `@Service` vs `@Repository`?

All are Spring component stereotypes.

- `@Component` → generic
- `@Service` → business logic
- `@Repository` → persistence layer + exception translation

Q16. `@Bean` vs `@Component`?

- `@Component` → put on the class; component scanning discovers it.
- `@Bean` → put on a method; the returned object becomes a Bean.

Q17. `@Value` vs `@ConfigurationProperties`?

- `@Value` → convenient for individual properties.

- `@ConfigurationProperties` → better for grouping related configuration.

Q18. What does `@Transactional` do?

It executes database-related work within a transaction managed by Spring.

Q19. What problem does Spring Boot solve?

It reduces configuration and setup using auto-configuration, starters, embedded servers, and sensible defaults.

Q20. Does Spring Boot replace Spring?

No.

Spring Boot simplifies the use of Spring Framework.

15. One-Minute Mental Model

